

Ingeniería en Computación

Curso: Taller de Programación



Profesor:

Diego Mora Rojas

Jeff Schmidt Peralta

Período: Primer Semestre 2026

NOMBRE DEL PROYECTO

Base Raiders

NOMBRE DEL ESTUDIANTE O INTEGRANTES

Estudiantes:

Elian Montero Valverde

Luis Carlos Fuentes Calvo

1. DESCRIPCIÓN GENERAL DEL JUEGO Y REGLAS

1.1 Mecánica de Rondas y Flujo

Base Raiders es un juego de estrategia asimétrico por turnos para dos jugadores. Un jugador asume el rol de Defensor y el otro el de Atacante. Las partidas se dividen estructuralmente en rondas consecutivas que siguen este orden estricto:

1. Fase de Suministro: Se asigna un presupuesto inicial de dinero a ambos jugadores.
2. Fase de Construcción (Defensor): El defensor gasta sus recursos colocando muros y torres defensivas sobre el mapa cuadrulado para proteger la Base Central (la cual posee una ubicación fija en la matriz).
3. Fase de Despliegue (Atacante): El atacante invierte su capital en la compra y posicionamiento de unidades de asalto en los extremos del mapa.
4. Fase de Combate: Se ejecuta la simulación de combate en la interfaz. Las unidades avanzan automáticamente por los caminos libres, mientras que las torres atacan dentro de su área de alcance.
5. Fase de Balance Económico: Al cerrar la ronda, el atacante gana dinero extra según el daño realizado a las estructuras y la base, mientras que el defensor gana dinero por cada unidad enemiga eliminada.

1.2 Condiciones de Victoria

- **De la Ronda (Defensor):** Gana la ronda si elimina a todas las unidades atacantes en el mapa o si el atacante se queda sin dinero para desplegar más tropas, manteniendo la Base Central en pie.
- **De la Ronda (Atacante):** Gana la ronda de manera inmediata si logra reducir los puntos de vida de la Base Central del defensor a 0.
- **De la Partida Completa:** El primer jugador en ganar 3 rondas es declarado ganador absoluto. Al finalizar, el sistema actualiza automáticamente el registro permanente con las nuevas victorias indexadas al perfil del usuario según el rol con el que ganó.

2. EXPLICACIÓN DETALLADA DE ENTIDADES Y FACCIONES

De acuerdo con el catálogo de imágenes dinámicas y módulos del código fuente (game/graphics/), el juego implementa las siguientes entidades con sus respectivas lógicas de habilidades (las cuales se ejecutan desde la clase lógica principal):

2.1 Estructuras del Defensor (game_logic.py)

- **Muro (Wall):** Estructura de costo muy bajo. No ataca, pero cuenta con alta vida para bloquear el avance físico de las tropas.
- **Torre de Arqueros (Crossbow Tower):** Daño normal a distancia y costo bajo. Su habilidad especial es el Disparo Doble (ataca a dos unidades simultáneamente cada 2 turnos).

- **Torre de Pinchos (Spiky Tower):** Mucha vida y daño alto, pero costo elevado. Su habilidad especial es el Daño en Área (afecta a todas las unidades en casillas adyacentes cada 3 turnos).
- **Torre de Magos (Wizard Tower):** Daño base bajo pero de utilidad táctica. Su habilidad especial es Congelar una Unidad (detiene por completo el movimiento de un enemigo durante 1 turno).

2.2 Unidades del Atacante (game_logic.py)

- **Duende (Goblin / Unidad Rápida):** Poco daño y vida baja, pero se desplaza velozmente (2 casillas por turno). Habilidad: Aumento de Velocidad temporal.
- **Caballero (Knight / Soldado Básico):** Costo balanceado y estadísticas estándar. Habilidad: Ataque Doble físico en su turno de activación.
- **Gigante / PEKKA / Dragón (Tanque):** Elevada cantidad de puntos de vida, pero movimiento lento. Habilidad: Escudo Temporal (reduce el daño recibido de las torres) o Daño Extra contra Torres.

2.3 Sistema de Facciones Visuales

El juego incluye 3 facciones distintas cuyos archivos visuales en formato .png cambian dinámicamente según la elección del jugador:

- **Medieval (_mdv):** Estilo clásico con castillos de piedra, empalizadas de madera y torres de vigía medievales.
- **Futurista (_ftr):** Estilo tecnológico con barreras de energía, torretas láser y bases de operaciones de plasma.
- **Naturaleza (_ntr):** Estilo orgánico con barreras de raíces, torretas botánicas y centros de comando de bosque.

Restricción: El Atacante y el Defensor no pueden usar la misma facción en la misma partida. Las ventajas se mantienen balanceadas ya que el cambio es primordialmente estético.

3. ARQUITECTURA DE SOFTWARE Y ESTRUCTURAS DE DATOS

3.1 Estructura Modular del Código Fuente

El proyecto se compone de un archivo principal en la raíz y subcapas lógicas que sobrepasan el mínimo de 3 clases requeridas:

- **main.py / game/main.py:** Punto de entrada que inicializa el bucle y las ventanas del juego.
- **game/classes.py:** Contiene las clases base y heredadas de las Torres, Muros y Unidades.
- **game/game_logic.py y game_loops.py:** Controlan las fases por turnos, detección de colisiones en el mapa, radios de alcance y ejecución de habilidades.
- **game/players_manager.py:** Clase encargada del registro de usuarios, login y actualización de estadísticas de victorias.

- **game/money_system.py y money_management.py:** Controlan las transacciones, validación de fondos y recompensas económicas de la ronda.
- **game/rankings_manager.py:** Filtra y procesa los datos de los usuarios para generar los rankings de rendimiento.
- **game/ui_windows.py / ui_screens.py / ui_functions.py:** Gestionan las vistas gráficas de Tkinter (Menú, Login, Mapa, Estadísticas).

3.2 Estructuras de Datos Utilizadas

- **Matriz Bidimensional (10x10):** Representada mediante listas de listas en Python (game_logic.py). Cada coordenada de la matriz contiene el objeto (Torre, Muro, Unidad) o un valor vacío para identificar los caminos libres.
- **Diccionarios (JSON / Hashmaps):** Utilizados para mapear los atributos de las entidades y para la gestión en memoria de los perfiles de usuario y sus contraseñas.
- **Listas (Arrays):** Utilizadas para almacenar y ordenar las colecciones de jugadores al generar el Top 5 mediante funciones de ordenamiento.

4. SISTEMA DE ARCHIVOS Y PERSISTENCIA (game_data/)

Para que los datos de los jugadores se conserven al cerrar la aplicación, el programa implementa un almacenamiento permanente basado en archivos de formato JSON:

- Archivo de datos: game/game_data/players.json.
- Estructura de persistencia en disco organizada en un diccionario raíz indexado por nombres de usuario, guardando contraseñas seguras y contadores asimétricos para victorias_defensor y victorias_atacante.

La clase PlayersManager lee este archivo mediante bloques de control try/except al arrancar el programa, y realiza una escritura masiva resguardando los datos cada vez que finaliza una partida completa.

5. MANUAL DE USUARIO E INSTRUCCIONES DE EJECUCIÓN

5.1 Requisitos del Sistema

- Python 3.10 o superior instalado en el sistema.
- Biblioteca Pygame (usada estrictamente para la música y efectos de sonido). Instalación via 'pip install pygame'.
- Tkinter: Viene preinstalado de forma estándar con Python.

5.2 Pasos para Ejecutar el Programa

1. Asegúrese de mantener la estructura completa descomprimida del archivo .zip (no ejecute archivos sueltos).
2. Abra una terminal o consola de comandos en la carpeta raíz del proyecto (Base-Raiders--main/).

3. Ejecute el archivo principal escribiendo: `python game/main.py`

5.3 Flujo de Uso en la Interfaz Gráfica

6. Registro/Login: En la pantalla inicial, registre dos usuarios distintos con sus respectivas contraseñas o inicie sesión con perfiles ya existentes.
7. Selección de Facciones: Escoja una facción (Medieval, Futurista o Naturaleza) para cada rol. El sistema validará que no sean iguales.
8. Fase de Juego: Use el ratón para hacer clic en la cuadrícula de 10x10. El defensor colocará estructuras usando sus botones interactivos y fondos de la ronda. Luego, el atacante posicionará sus tropas. Presione 'Iniciar Turno/Combate' para ver la resolución de la batalla con música y efectos sonoros activos.
9. Consulta de Rankings: Desde el Menú Principal, presione el botón de Estadísticas para abrir la ventana con el Top 5 de Defensores y el Top 5 de Atacantes ordenados de mayor a menor número de victorias.

6. CONCLUSIONES INDIVIDUALES

Conclusión de Luis:

Concluyó que este proyecto me retó y me guió a aprender materia en lo relacionado a la programación: la división de proyectos en varias carpetas y archivos, el trabajo en equipo, el uso de github —de lo más desafiante del proyecto—, el manejo de archivos JSON, entre otros. He de admitir que me hubiera gustado elaborar más en la jugabilidad y la interfaz gráfica, pero no me disgusta para nada nuestro resultado. Fue divertido hacer la lógica del juego.

Conclusión de Elian:

Considero que este proyecto fue una experiencia muy positiva y una de las que más disfruté durante el curso. Me gustó mucho trabajar en parejas, ya que la carga de trabajo no recaía únicamente en una persona y siempre era posible apoyarse en el compañero en caso de tener dudas o enfrentar algún problema. Esto hizo que el desarrollo fuera más llevadero y permitió aprender de manera conjunta.

A lo largo del proyecto también surgieron algunas dificultades, principalmente relacionadas con el uso de GitHub. En varias ocasiones se mezclaron proyectos o cambios que no tenían relación entre sí, y algunas veces tuve problemas al realizar los pull, lo que ocasionó ciertos inconvenientes al trabajar con el repositorio. Sin embargo, estas situaciones también me permitieron aprender más sobre el manejo de control de versiones y comprender la importancia de mantener una buena organización al trabajar en equipo.

En general, fue una experiencia que disfruté mucho, tanto por los conocimientos adquiridos como por la oportunidad de trabajar de forma colaborativa y desarrollar un proyecto más completo de lo que habría sido posible de manera individual.

7. ENLACES DEL PROYECTO

- Repositorio de GitHub: <https://github.com/tu-usuario/Base-Raiders--main> (Muestra una progresión natural e historial de commits acorde al desarrollo continuo del proyecto).
- Video Evidencia de Desarrollo (10 Minutos):
https://drive.google.com/drive/folders/1ju_AcXpuWgTwGIDkVewxOqA25y-1vRRB?usp=sharing